

이화여자대학교 컴퓨터공학과

2026학년도 1학기

캡스톤디자인과 창업프로젝트B 그로쓰 2차 보고서

대학생 초보 개발 팀의 프로젝트 회의 기록과 일정 관리를 지원하는 AI 기반 서비스



트랙	산학
팀명	03 세얼간이
팀원	정서윤(2276278) 강민지(2376003) 임이랑(2371052)
지도교수	오세은 교수님

Team Info

과제명	KNOTE - 대학생 초보 개발 팀의 프로젝트 회의 기록과 일정 관리를 지원하는 AI 기반 서비스
팀 정보	03 세얼간이
팀 구성원	정서윤(2276278, 팀장, BE) 강민지(2376003, BE) 임이랑(2371052, FE)

문제정의

개요

KNOTE는 대학생 초보 개발 팀의 프로젝트 회의 기록과 일정 관리를 지원하는 AI 기반 서비스이다.

문제 배경

현재 팀 프로젝트는 직장 사회 뿐만 아니라 대학 교육 과정에서도 거의 필수적으로 요구되는 학습 활동이다. 팀 프로젝트를 통해 학생은 개인이 혼자 해결하기 어려운 크기의 문제를 해결하고 성취감과 협동력을 기르는 효과가 있으나, 실제로 학교 현장에서는 90%가 넘는 학생이 ‘팀 프로젝트에서 심한 스트레스를 느낀 적 있다’고 응답¹한다. 이토록 심한 스트레스를 느끼는 이유로 학생들은 일정 조율의 어려움, 역할 불균형 등을 이유로 꼽는다.

타겟 사용자

따라서 이 서비스는 다음과 같은 3가지의 사용자 그룹을 타겟으로 한다.

1. 대학생
2. 초보 개발자
3. 소규모 프로젝트

이 그룹은 대학생에서 시작하여, 유기적으로 설정된 사용자 그룹이다. 대다수의 대학생은 개발을 처음 시작하는 초보 개발자이며, 학부 수준의 초보 개발자들이 협력하여 프로젝트를 처리하기 때문에 프로젝트의 규모를 소규모로 예측하고 타겟 사용자를 설정하였다.

¹ <https://news.gknu.ac.kr/news/articleView.html?idxno=1802>

문제점(Pain Point)

이를 토대로 사용자 그룹이 다음과 같은 Pain Point를 보유하고 있다고 파악하였다.

1. 회의 기록 체계성이 부족하다
2. 회의 후 내용이 task로 연결되지 않는다
3. 팀원 간 업무 분배가 모호하다
4. 사용자가 피드백, 태도를 스스로 점검하기 어렵다

기존 서비스와의 비교

기존 협업 도구는 회의와 업무가 분리되어 관리되는 반면, 본 서비스는 회의 데이터를 기반으로 업무 생성과 프로젝트 관리까지 자동으로 연결하는 협업 관리 시스템이다.

즉, 회의 → Task → 프로젝트 관리 → 협업 분석까지 이어지는 통합 시스템이라는 점에서 기존 서비스와 차별성을 가진다. 구체적인 차별점은 다음과 같다.

1. Meeting → Task 자동 전환

<기존 서비스>

- Notion: 사용자가 직접 To-Do 리스트 작성
- Jira: Issue(Task)를 사용자가 직접 생성
- Slack / Zoom: 회의 및 대화 내용은 기록되지만 → 업무(Task)로 자동 연결되지 않음

∴ 회의 내용 → 업무(Task) 전환이 수동으로 이루어짐

<본 서비스 (KNOTE)>

- 회의 녹음 및 STT 기반 텍스트 변환된 회의 내용을 AI가 분석하여 Task 자동 추출
- 발화자를 기반으로 담당자 자동 매칭
- 프로젝트 단계에 따라 업무 분리 및 구조화

∴ 회의 → Task 전환을 자동화하여 업무 생성 과정의 비효율 제거

2. 진행 과정 데이터 기반 협업 관리

<기존 서비스>

- Jira: Task 상태 기반 관리 (To Do / In Progress / Done)
- Notion: 문서 및 일정 중심 관리
- Slack / Zoom: 커뮤니케이션 중심

∴ 회의 내용과 프로젝트 진행 정보가 분리되어 관리됨

<본 서비스 (KNOTE)>

- 회의 내용을 기반으로 개발 과정을 5단계로 분리, 현재 단계에 해당하는 업무 추천
- 회의 내용을 중심으로 업무(Task)와 프로젝트 진행을 통합 관리

∴ 회의 데이터를 기반으로 어떤 업무가 생성되었는지, 어떻게 수행되고 있는지, 프로젝트가 어떻게 진행되는지를 하나의 흐름으로 관리함

3. 피드백, 회고

<기존 서비스>

- 프로젝트 데이터만 제공
- 팀원 간 피드백: 과정 중 정확한 피드백을 나누기 어려움, 팀원에 따라 주관적임

∴ 객관적인 실시간 피드백을 받기 어려움

<본 서비스 (KNOTE)>

- 회의 데이터와 업무 수행 데이터를 동시에 고려한 AI 기반 협업 피드백을 제공함
 - 회의 참여율, 발화량, 업무 수행률, 주간 진행 상태를 종합적으로 분석

∴ 데이터를 기반으로 사용자에게 객관적인 피드백을 제공함

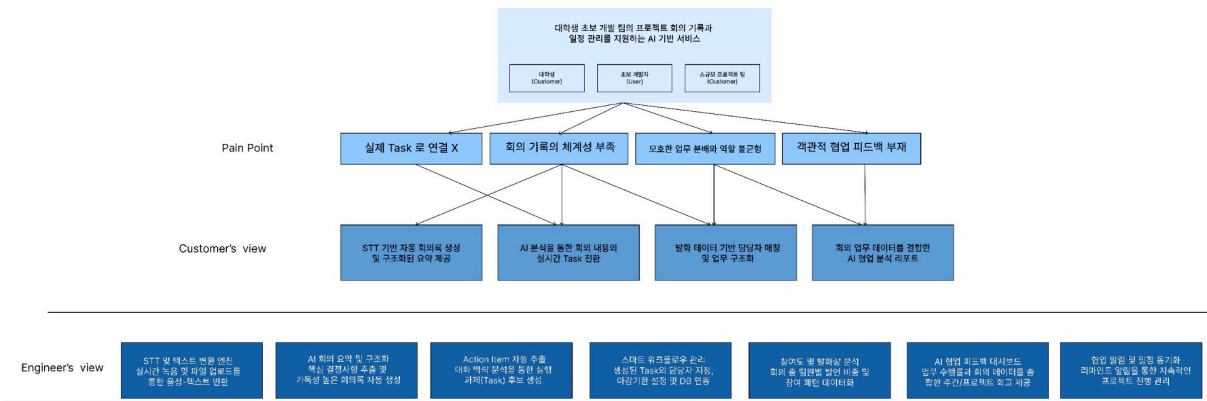
4. 정리

	기존 서비스	KNOTE
Todo 생성	회의 내용 → 업무(Task) 전환이 수동으로 이루어짐 ● Notion: 사용자가 직접 To-Do 리스트 작성 ● Jira: Issue(Task)를 사용자가 직접 생성 ● Slack / Zoom: 회의 및 대화 내용은 기록되지만 업무(Task) 정리는 사용자가 직접 해야 함	● 회의 녹음 및 STT 기반 텍스트 변환된 회의 내용을 AI가 분석하여 Task 자동 추출 ● 발화자를 기반으로 담당자 자동 매칭 ● 프로젝트 단계에 맞게 업무 분리
프로젝트 진행 관리	회의 내용과 프로젝트 진행 정보가 분리되어 관리됨 ● Jira: Task 상태 기반 관리 (To Do / In Progress / Done) ● Notion: 문서 및 일정 중심 관리 ● Slack / Zoom: 커뮤니케이션 중심	● 개발 과정을 5단계로 분리, 회의 내용을 기반으로 각 단계에 해당하는 업무 추천 ● 회의 내용 - 프로젝트 진행 - 업무 관리가 통합되어 관리됨
피드백, 회고	객관적인 실시간 피드백을 받기 어려움 ● 프로젝트 데이터만 제공, 과정에 대한 분석 없음 ● 팀원에게 면대면으로 피드백 받아야 함 ○ 팀원에 따라 피드백이 주관적	● 회의 데이터와 업무 수행 데이터를 모두 고려해 AI 기반 피드백을 제공 ● 프로젝트 진행 중에도 자신의 상태를 객관적으로 점검할 수 있음

제안 내용

회의 - 업무 - 결과물까지 연결되는 프로젝트 관리 환경을 제시하기 위해, 각 Pain Point에 대해 다음과 같은 solution을 제시하기로 하였다.

- 1. 회의 기록 체계성이 부족한 문제
: STT 기반 회의 기록 및 자동 요약
 - 사용자가 서비스 내에서 회의를 녹음하거나 서비스 외부에서 녹음한 파일을 업로드하면 STT를 통해 회의록을 대신 기록한다.
 - 자동 기록된 회의록을 바탕으로 AI가 회의 내용을 분석하여 핵심 회의 내용을 자동 요약해 제공한다.
 - 회의 내용을 구조화하여 이후 사용자가 활용 가능한 형식으로 저장한다.
- 2. 회의 후 내용이 task로 연결되지 않는다
: AI 회의 분석 및 Task 생성
 - AI가 회의 요약 내용을 분석하여 프로젝트 진행에 필요한 task를 생성한다.
- 3. 팀원 간 업무 분배가 모호한 문제
: 회의 기반 task 생성 및 담당자 매칭
 - 2번 해결책에서 AI가 추출한 task는 저장된 회의록을 토대로 관련 발언을 한 사람, 기존에 관련 업무를 담당하던 사람에게 매칭된다
- 4. 사용자가 피드백, 태도를 스스로 점검하기 어렵다
: AI 피드백 제공
 - 회의 데이터(회의 참여율, 발화량)와 task(또는 todo) 수행 데이터를 분석하여 협업 상태에 대한 AI 피드백을 제공한다.
 - 프로젝트 종료 후 팀원 간 회고 뿐만 아니라 프로젝트 진행 중에도 사용자가 태도를 개인적, 객관적으로 점검할 수 있도록 도와준다.



<그림 설명>

1. 목표: 대학생 및 초보 개발 팀을 위한 AI 기반 회의-업무 통합 관리 시스템 구축
Customer(대학생), User(초보 개발자), Customer(소규모 프로젝트 팀)

2. Sub-problem (Pain Points)

- Sub-problem 1: 회의 기록의 체계성 부족 및 수기 작성의 번거로움
- Sub-problem 2: 회의 내용이 실제 실행 가능한 Task로 연결되지 않음
- Sub-problem 3: 팀원 간 모호한 업무 분배와 역할 불균형
- Sub-problem 4: 객관적인 협업 태도 점검 및 피드백 부재

3. Solution (Customer View)

- Solution 1: STT 기반 자동 회의록 생성 및 구조화된 요약 제공
- Solution 2: AI 분석을 통한 회의 내용의 실시간 Task 전환
- Solution 3: 발화 데이터 기반 담당자 매칭 및 업무 구조화
- Solution 4: 회의/업무 데이터를 결합한 AI 협업 분석 리포트

4. 기능 (Engineer View / Features)

구분	주요 기능	상세 내용
기능1	STT 및 텍스트 변환 엔진	실시간 녹음 및 파일 업로드를 통한 음성-텍스트 변환
기능 2	AI 회의 요약 및 구조화	핵심 결정사항 추출 및 가독성 높은 회의록 자동 생성
기능 3	Action Item 자동 추출	대화 맥락 분석을 통한 실행 과제(Task) 후보 생성
기능 4	스마트 워크플로우 관리	생성된 Task의 담당자 지정, 마감기한 설정 및 db 연동
기능 5	참여도 및 발화량 분석	회의 중 팀원별 발언 비중 및 참여 패턴 데이터화
기능 6	AI 협업 피드백 대시보드	업무 수행률과 회의 데이터를 종합한 주간/프로젝트 회고 제공
기능 7	협업 알림 및 일정 동기화	리마인드 알림을 통한 지속적인 프로젝트 진행 관리

기대 효과 및 의의

KNOTE는 기존 협업 도구에서 분리되어 관리되던 회의, 업무, 피드백 과정을 하나의 흐름으로 통합함으로써 대학생 초보 개발 팀이 흔히 겪을 수 있는 시행착오와 비효율을 다음과 같이 구조적으로 개선할 수 있다.

1. 회의 기록 자동화 및 정보 활용성 향상
 - STT 기반 자동 회의 기록과 AI 요약 기능을 통해 회의 내용을 별도로 정리해야 하는 부담을 줄이고, 회의 기록의 누락이나 왜곡을 최소화할 수 있다.
 - 사용자 팀은 회의를 진행하는 동안 기록 등의 부가 업무를 배제하고 회의에만 집중할 수 있으며, 이후에도 필요할 경우 정리된 핵심 내용을 빠르게 파악할 수 있다.
2. 회의-업무 연결을 통한 프로젝트 연속성 확보
 - 회의 내용 기반의 Task 자동 생성을 통해 회의에서 논의된 내용이 실제 업무로 이어지지 않는 문제를 해결할 수 있다.
 - 기존 서비스에는 사용자가 직접 회의록을 기반으로 업무를 정리해야 하는 번거로움이 존재하나, 본 서비스에서는 회의 → 업무 전환이 자동화되어 업무 생성 과정의 누락과 비효율을 줄이고 프로젝트 진행의 연속성을 확보할 수 있다.
3. 데이터 기반 업무 분배로 협업 균형 개선
 - 발화 데이터 및 기존 업무 이력을 기반으로 담당자를 매칭하는 기능은 팀 내 역할 분배의 모호함을 감소시켜 합리적인 업무 배분을 가능하게 한다.
 - 특정 인원에게 업무가 편중되는 문제를 완화하고 팀원 간 협업의 균형을 유지할 수 있다.
4. 객관적 협업 피드백을 통한 과정 중심 개선
 - 회의 참여도, 발화량, 업무 수행률 등의 데이터를 기반으로 제공되는 AI 피드백 기능은 객관적인 협업 평가 기준을 제시한다.
 - 사용자는 프로젝트 진행 과정에서 자신의 협업 태도와 기여도를 스스로 점검할 수 있으며, 결과 중심이 아닌 과정 중심의 협업 개선을 가능하게 한다.

즉, KNOTE는 회의 기록 → 업무 생성 → 프로젝트 진행 → 협업 피드백으로 이어지는 일관되며 지속적인 협업 관리 흐름을 제공함으로써 대학생 초보 개발 팀이 보다 효율적으로 프로젝트를 수행할 수 있도록 지원한다. 이는 단순한 기능 제공을 넘어, 회의 - 업무 - 결과물까지 연결되는 프로젝트 관리 환경을 제공한다는 의의를 가진다.

주요 기능 리스트

1. AI 피드백 제공

1-1. 회의 단위 & 주간 단위 협업 피드백 생성

회의별 데이터와 일정 기간 동안의 누적 데이터를 함께 활용하여, 단기적인 회의 단위 피드백과 장기적인 주간 단위 피드백을 동시에 제공한다. 이를 통해 팀은 개별 회의의 개선점뿐 아니라 전체 프로젝트 흐름 속 협업 상태를 함께 점검할 수 있다.

1-2. 회의 녹음 데이터와 투두 이수율 기반 분석

회의에서 생성된 녹음 데이터(STT 결과)와 해당 회의에서 도출된 Task의 완료 여부를 함께 분석하여, 실제 논의 내용이 실행으로 얼마나 이어졌는지를 평가한다. 이를 통해 단순 참여가 아닌 실행 중심의 협업 수준을 판단할 수 있다.

1-3. 회의 참여율·발화량·업무 수행률을 종합 평가

각 팀원의 발언 비율, 회의 참여 빈도, 할당된 업무의 수행률을 종합적으로 분석하여 협업 기여도를 평가한다. 이를 기반으로 팀 내 참여 편차나 특정 인원에게 업무가 집중되는 문제를 파악할 수 있다.

1-4. 회의별 피드백 + 주간 진행 상황을 통해 협업 상태 점검 지원

회의 단위에서 도출된 피드백과 주간 단위의 프로젝트 진행 데이터를 함께 제공하여, 팀이 현재 협업 상태를 입체적으로 이해할 수 있도록 지원한다. 이를 통해 지속적인 개선 방향을 설정하고 프로젝트 진행의 효율성을 높일 수 있다.

2. AI 기반 회의 분석

2-1. 회의 내용을 화자별로 정리한 요약 제공

회의에서 수집된 음성 데이터를 텍스트로 변환한 후, 화자 단위로 발언을 구분하여 구조화된 요약을 제공한다. 이를 통해 회의 내용을 빠르게 파악할 수 있으며, 발언 주체와 맥락을 함께 이해할 수 있다.

2-2. 회의에서 필요한 Task 자동 To-Do 추출

회의 내용 중 실행이 필요한 항목을 AI가 분석하여 Task 형태로 자동 추출한다. 이를 통해 사용자는 별도의 정리 과정 없이도 바로 업무 목록을 확인하고 관리할 수 있다.

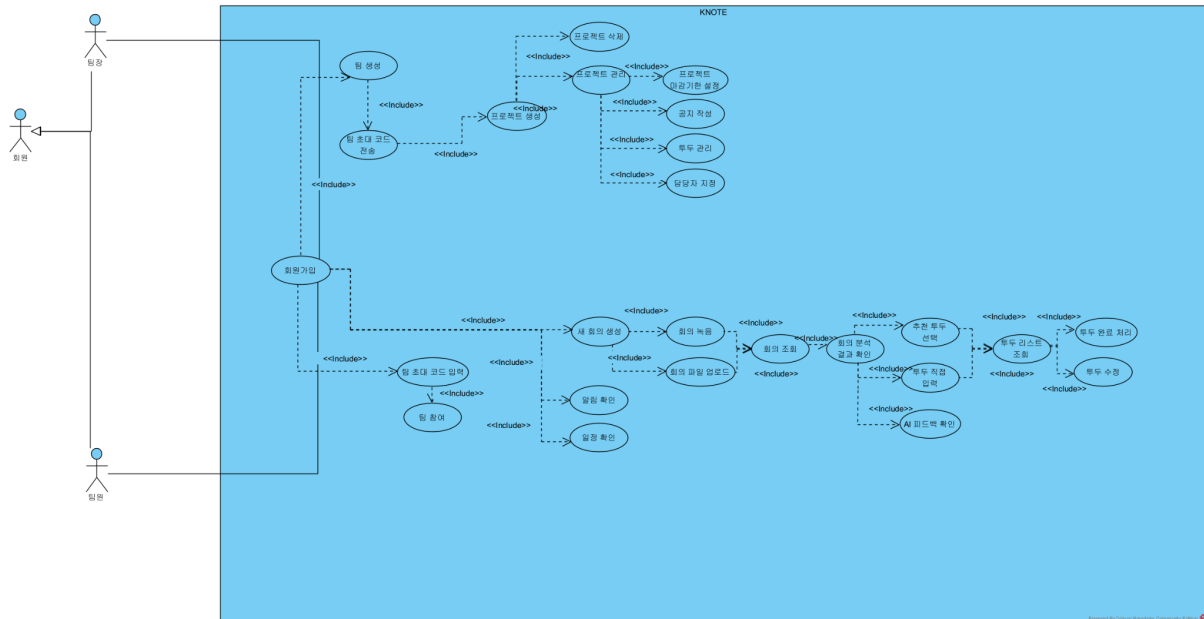
2-3. 발언 화자를 기반으로 담당자 자동 매칭

Task와 관련된 발언을 한 화자를 기반으로 해당 업무의 담당자를 자동으로 매칭한다. 이를 통해 업무 분배 과정에서 발생할 수 있는 혼선을 줄이고, 자연스러운 역할 분담을 유도한다.

2-4. 프로젝트 진행 단계에 따른 업무 분리 및 일정 제안

회의 내용을 분석하여 현재 프로젝트의 진행 단계를 파악하고, 그에 맞는 업무를 분리 및 정리한다. 또한 Task의 우선순위와 흐름을 고려하여 적절한 일정 및 마감 기한을 제안함으로써 체계적인 프로젝트 관리를 지원한다.

요구사항 정의



본 서비스는 도출된 Pain Point를 기반으로 다음과 같은 요구사항을 정의하였다. 사용자와 시스템 간의 상호작용은 UML 유스케이스 다이어그램을 통해 확인할 수 있다.

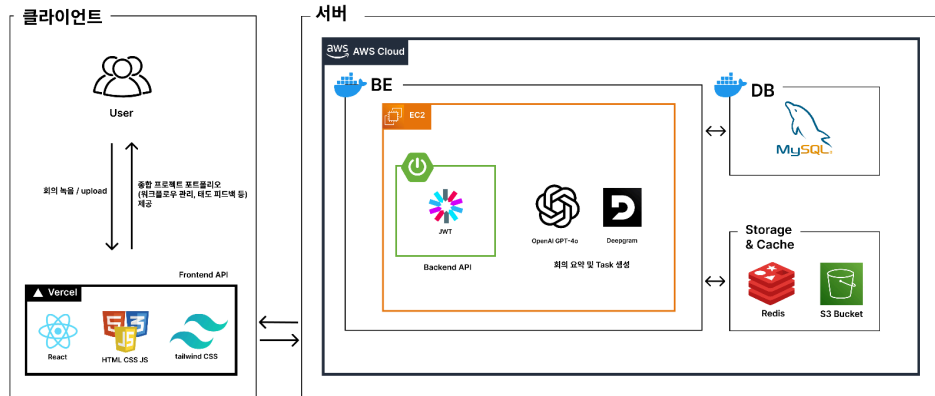
1. 회의 기록 체계성 부족 해결
 - 사용자가 회의를 생성하거나 녹음 파일을 업로드하면 텍스트로 자동 변환되어야 한다.
 - 변환된 회의 내용은 요약 및 구조화된 형태로 제공되어야 한다.
 - 이를 위해 시스템은 회의 생성, 회의 녹음/파일 업로드, 회의 조회, 회의 분석 결과 확인 기능을 제공해야 한다.
2. 회의 → Task 연결 자동화
 - 회의 분석 결과를 기반으로 수행해야 할 업무(Task)가 도출되어야 한다.
 - 사용자는 추천된 Task를 확인하고 실제 업무로 관리할 수 있어야 한다.
 - 이를 위해 시스템은 회의 분석 결과 확인, 추천 Task 확인, 액션 아이템 관리 기능을 제공해야 한다.
3. 업무 분배 명확화 및 프로젝트 관리
 - 생성된 업무에 대해 담당자를 지정하고 마감기한을 설정할 수 있어야 한다.
 - 팀 및 프로젝트 단위로 업무를 구조적으로 관리할 수 있어야 한다.
 - 이를 위해 시스템은 팀 생성, 팀 초대, 프로젝트 생성 및 관리, 담당자 지정, 마감기한 설정 기능을 포함해야 한다.
4. 협업 피드백 및 자기 점검 지원
 - 회의 데이터와 업무 수행 데이터를 기반으로 협업 상태를 분석한 피드백이 제공되어야 한다.
 - 사용자는 자신의 참여도 및 업무 수행 상태를 지속적으로 확인할 수 있어야 한다.

- 이를 위해 시스템은 AI 피드백 확인, 프로젝트 진행 현황 확인, 일정/알림 확인 기능을 제공해야 한다.

5. 통합 협업 흐름 제공

- 회의 생성 → 회의 분석 → 업무(Task) 관리 → 협업 피드백으로 이어지는 일관된 흐름이 제공되어야 한다.

전체시스템 구성



- 시스템 구조도 수정

1) Frontend

- HTML, JavaScript
- Tailwind CSS

프론트엔드는 사용자가 회의 생성, 파일 업로드, 회의 요약 확인, 액션 아이템 조회 및 협업 기능을 직관적으로 사용할 수 있도록 웹 화면을 구성하는 역할을 담당한다.

특히 UI 퍼블리싱을 위해 Tailwind CSS를 사용하여 반복적인 스타일 정의를 줄이고, 컴포넌트 단위로 일관된 화면 구성을 빠르게 구현할 수 있도록 하였다. 이를 통해 회의, 업무, 팀 관리 화면을 효율적으로 설계하고 반응형 UI 구성에도 유연하게 대응할 수 있다.

2) Backend

- Java
- Spring Framework / Spring Boot
- JPA, Hibernate

백엔드는 서비스의 핵심 비즈니스 로직을 처리하는 영역으로, 회원 관리, 팀 관리, 회의 생성, 파일 업로드, STT 처리 요청, AI 분석 결과 저장, 액션 아이템 관리 등의 기능을 담당한다.

서버 프레임워크로는 Spring Boot를 사용하여 REST API를 구현하고, 유지보수성과 확장성을 고려한 계층형 구조로 개발하고자 하였다.

또한 JPA와 Hibernate를 활용하여 객체와 데이터베이스 간 매핑을 처리함으로써 생산성을 높이고, 복잡한 데이터 관계를 보다 효율적으로 관리할 수 있도록 하였다.

3) Database

- MySQL

데이터베이스는 회원, 팀, 프로젝트, 회의, STT 결과, 회의 요약, 액션 아이템, 알림 등의 관계형 데이터를 저장하기 위해 MySQL을 사용한다.

본 프로젝트는 팀-프로젝트-회의-업무 구조처럼 엔티티 간 관계가 명확하므로, 관계형 데이터베이스를 통해 데이터 무결성과 일관성을 유지하도록 설계하였다.

4) 인증 및 세션 관리

- JWT (Json Web Token)
- Redis

서비스 사용자 인증을 위해 JWT 기반 인증 방식을 사용하며, 토큰 관리 및 보안 강화를 위해 Redis를 함께 활용한다.

사용자가 로그인하면 서버는 Access Token과 Refresh Token을 발급하며, Access Token은 클라이언트 측에서 인증에 사용되고, Refresh Token은 Redis에 저장하여 관리한다.

Redis는 인메모리 기반 Key-Value 저장소로, 빠른 데이터 처리 속도를 제공하므로 토큰 검증 및 재발급 과정에서 효율적으로 활용할 수 있다. 특히 Redis의 TTL(Time-To-Live) 기능을 활용하여 Refresh Token의 만료 시간을 자동으로 관리하도록 설계하였다.

또한 로그아웃 시 Redis에 저장된 Refresh Token을 삭제함으로써 토큰을 즉시 무효화할 수 있으며, 이를 통해 JWT의 무상태 인증 구조에서 발생할 수 있는 보안 문제를 보완하였다.

이와 같은 구조를 통해 인증 처리 성능을 유지하면서도, 토큰 재발급, 만료 관리, 로그아웃 처리 등 인증 제어 기능을 효과적으로 구현하였다.

5) STT 처리

- ~~Whisper STT~~

~~회의 녹음 파일을 텍스트로 변환하기 위한 STT 기술로 Whisper STT를 활용할 예정이다.~~

~~Whisper는 음성 데이터를 문장 단위 텍스트로 변환하는 데 활용되며, 회의 원문 확보를 통해 이후 요약, 핵심 결정사항 추출, 액션 아이템 생성 등의 후속 AI 분석 기능의 기반 데이터를 제공한다.~~

~~이를 통해 사용자는 회의 내용을 직접 다시 듣지 않아도 텍스트 기반으로 회의 기록을 확인할 수 있다.~~

- Deepgram STT (Diarization)

회의 녹음 파일을 정교하게 텍스트로 변환하고 화자를 식별하기 위해 Deepgram STT 기술을 활용한다. Deepgram은 단순 음성-텍스트 변환을 넘어, 다중 화자가 참여하는 회의 환경에 최적화된 화자 분리(Diarization) 기능을 제공하여 각 발언자가 누구인지 실시간으로 구분한다.

확보된 화자별 텍스트 데이터는 이후 진행되는 AI 요약, 핵심 결정사항 추출, 액션 아이템 생성의 정확도를 높이는 핵심 기반이 된다. 이를 통해 사용자는 발언 주체와 내용이 명확히 구분된 회의록을 확인할 수 있으며, 기록의 가독성과 업무 추적성을 극대화할 수 있다.

6) AI 분석

- OpenAI GPT-4o

STT를 통해 변환된 회의 텍스트를 분석하기 위해 OpenAI GPT-4o를 활용할 예정이다.

GPT-4o는 회의 내용을 바탕으로 요약 생성, 핵심 결정사항 정리, 액션 아이템 도출, 협업 피드백 생성과 같은 기능에 사용된다.

이를 통해 단순 기록 저장에 그치지 않고, 회의 데이터를 실제 협업과 업무 수행에 활용할 수 있는 구조로 확장하고자 하였다.

7) 배포 및 운영 관리

- AWS EC2
- AWS S3 Bucket
- GCP (Google Cloud Platform)

배포 및 운영 환경으로는 AWS와 GCP를 함께 활용할 예정이다.

우선 서비스 운영 서버는 AWS EC2를 통해 배포하여 백엔드 애플리케이션을 안정적으로 실행하고자 하며, 회의 녹음 파일이나 첨부 파일과 같은 정적 파일 및 대용량 파일 저장을 위해 AWS S3 Bucket을 사용할 계획이다.

이를 통해 애플리케이션 서버와 파일 저장소를 분리하여 관리 효율성과 확장성을 높이하고자 한다.

또한 초기 개발 및 테스트 단계에서 사용하는 mock 서버(Mock Server)는 GCP 환경에 배포하여 기능 검증 및 API 연동 테스트를 수행할 예정이다.

주요 엔진 및 기능 설계

1. STT 처리 엔진

역할

- Deepgram Nova-2 모델을 활용하여 음성 파일을 텍스트로 변환
- 화자 식별(Diarization)

핵심 코드

DeepgramService의 `transcribeWithDiarization`

설명

REST API를 통해 비동기로 음성 데이터를 전송하고, 화자 레이블(`Speaker 0, 1...`)이 포함된 구조화된 JSON 응답을 파싱하여 회의 원문을 생성

```
public String transcribeWithDiarization(byte[] fileBytes) {
    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.parseMediaType("audio/wav")); // 파일 형식에
    맞춰 변경 가능
    headers.set("Authorization", "Token " + apiKey);

    // 화자 분리(diarize), 한국어(ko), 스마트형식(smart_format) 옵션 추가
    String url = apiUrl +
        "?model=nova-2&smart_format=true&diarize=true&language=ko";

    HttpEntity<byte[]> requestEntity = new HttpEntity<>(fileBytes, headers);

    try {
        ResponseEntity<Map> response = restTemplate.postForEntity(url, requestEntity,
            Map.class);
        return formatDeepgramResult(response.getBody());
    } catch (Exception e) {
        log.error("Deepgram API 호출 실패", e);
        throw new RuntimeException("화자 분리 처리 중 오류 발생");
    }
}
```

2. AI 회의 분석 엔진

역할

- GPT-4o-mini 모델을 활용하여 원문에서 맥락을 파악, 요약 및 결정사항 추출

핵심 코드

OpenAiService의 summarize

설명

고도화된 System Prompt를 통해 단순 요약을 넘어 회의의 성격(결정사항, 논의 배경 등)을 구분하여 정형화된 JSON 포맷으로 반환

OpenAiService.java

```
// ChatGPT 요약 호출
public String summarize(String transcript) {
    String url = baseUrl + "/chat/completions";

    // 1. 위에서 정의한 고도화된 프롬프트
    String systemPrompt =
        "너는 전문 회의 기록가이자 프로젝트 매니저야. 제공된 회의 스크립트를 분석해서 'Action Items(할 일)'를 추출해.\n\n" +
        ...

    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_JSON);
    headers.setBearerAuth(apiKey);

    // 2. body 구성 시 'content' 부분에 위에서 만든 systemPrompt 변수 넣기
    Map<String, Object> body = new HashMap<>();
    body.put("model", "gpt-4o-mini");
    body.put("messages", List.of(
        Map.of("role", "system", "content", systemPrompt), // 여기서 변수 적용!
        Map.of("role", "user", "content", transcript)
    ));

    // 3. JSON 모드로 응답받기 위한 설정 (옵션: 모델이 JSON 형식을 더 잘 지키게 함)
    body.put("response_format", Map.of("type", "json_object"));

    HttpEntity<Map<String, Object>> requestEntity = new HttpEntity<>(body,
        headers);

    try {
        Map<String, Object> response = restTemplate.postForObject(url, requestEntity,
            Map.class);
        List<Map<String, Object>> choices = (List<Map<String, Object>>)
            response.get("choices");
        Map<String, Object> message = (Map<String, Object>)
```

```

choices.get(0).get("message");
    return (String) message.get("content");
} catch (Exception e) {
    log.error("GPT 요약 호출 중 에러 발생: ", e);
    return "{\"error\": \"요약 생성 실패\"}";
}
}

```

3. Action Item 생성 엔진

역할

- 회의 내용 중 실행 가능한 과업(TODO)을 식별
- 기한(Due Date)과 적정 단계(Phase) 추천

핵심 코드

summarize 내의 `actionItems` 추출 로직 및 `ActionItemService`

설명

AI가 할 일의 시급성을 판단하여 마감 기한을 제안하고, 개발 생명주기(SDLC)에 근거하여 5단계(Ideation ~ Launch) 중 적절한 Phase를 자동 할당한다.

MeetingService.java

```

// 4. ActionItem(AI To_do) 추출 및 저장
JsonNode actionItemsNode = root.path("actionItems");
if (actionItemsNode.isArray()) {
    for (JsonNode node : actionItemsNode) {
        ActionItem item = new ActionItem();

        // AI가 추출한 값
        item.setTitle(node.path("title").asText("새로운 할 일"));
        item.setMemo(node.path("memo").asText("내용 없음"));

        item.setDueDate(node.path("dueDate").isMissingNode()
            ? LocalDateTime.now().plusDays(3) // AI 제안이 없으면 기본 3일 뒤
            : LocalDateTime.parse(node.path("dueDate").asText()));

        // 도메인 제약조건(nullable=false)에 따른 필수 기본값 세팅
        item.setMeeting(meeting);
        item.setCompleted(false);
        item.setActivated(true);
        item.setIsConfirmed(true); // AI가 생성한 것이므로 일단 true 혹은 false
    }
}

```

```

        item.setDueDate(LocalDateTime.now().plusDays(3)); // 기본 마감기한 3일 뒤
        item.setPhase(node.path("phase").asLong(1L)); // 기본 단계
        item.setScope("TEAM"); // 기본 범위
        item.setImage("default.png"); // 기본 이미지

        // 담당자(assigneeMember)는 nullable이므로 세팅하지 않아도 됨

        actionItemRepository.save(item);
    }
}

```

4. 담당자 매칭 로직

역할

- AI가 제안한 데이터에 사용자의 판단을 결합, 최종 담당자를 매칭 및 데이터를 확정

핵심 코드

MeetingService의 `confirmActionItems` 및 ActionItemService의 `patchActionItem`

설명

AI의 제안(Suggested Speaker)을 참고하여 사용자가 실제 팀원을 매핑하고, PATCH API를 통해 제목 및 기한을 최종 수정함으로써 데이터의 신뢰성을 보장한다.

```

// 액션아이템 - 사용자 매칭
@Transactional
public void confirmActionItems(Long meetingId, List<ActionItemUpdateRequest>
requests) {
    // 해당 회의의 모든 액션 아이템을 가져옴
    for (ActionItemUpdateRequest req : requests) {
        ActionItem item = actionItemRepository.findById(req.getId())
            .orElseThrow(() -> new NotFoundException("해당 할 일을 찾을 수
없습니다."));

        // 1. 사용자가 체크박스를 선택했는지 여부 저장
        // (UI 기획상 선택 안 한 것도 가져가지만 상태값이 필요하다면 활용)
        item.setIsConfirmed(req.getIsSelected());

        // 2. 담당자 매칭 (assigneeId가 있을 경우에만)
        if (req.getAssigneeId() != null) {
            Member member = memberRepository.findById(req.getAssigneeId())
                .orElseThrow(() -> new NotFoundException("담당자를 찾을 수 없습니다."));

```

```

        item.setAssigneeMember(member);
    }

    actionItemRepository.save(item);
} // 액션아이템 - 사용자 매칭
@Transactional
public void confirmActionItems(Long meetingId, List<ActionItemUpdateRequest>
requests) {
    // 해당 회의의 모든 액션 아이템을 가져옴
    for (ActionItemUpdateRequest req : requests) {
        ActionItem item = actionItemRepository.findById(req.getId())
            .orElseThrow(() -> new NotFoundException("해당 할 일을 찾을 수
없습니다."));

        // 1. 사용자가 체크박스를 선택했는지 여부 저장
        // (UI 기획상 선택 안 한 것도 가져가지만 상태값이 필요하다면 활용)
        item.setIsConfirmed(req.getIsSelected());

        // 2. 담당자 매칭 (assigneeId가 있을 경우에만)
        if (req.getAssigneeId() != null) {
            Member member = memberRepository.findById(req.getAssigneeId())
                .orElseThrow(() -> new NotFoundException("담당자를 찾을 수 없습니다."));
            item.setAssigneeMember(member);
        }

        actionItemRepository.save(item);
    }

    // 3. 회의 상태를 최종 완료로 변경
    Meeting meeting = meetingRepository.findById(meetingId)
        .orElseThrow(() -> new NotFoundException("회의를 찾을 수 없습니다."));
    meeting.setStatus("COMPLETED");
}

// 3. 회의 상태를 최종 완료로 변경
Meeting meeting = meetingRepository.findById(meetingId)
    .orElseThrow(() -> new NotFoundException("회의를 찾을 수 없습니다."));
meeting.setStatus("COMPLETED");

}

```

ActionItemService.java 의 patchActionItem 메서드

```
// 스프린트 페이지/화면에서 제목이나 마감 기한을 실시간으로 수정할 때 사용
public void patchActionItem(Long id, ActionItemPatchRequest request) {
    ActionItem actionItem = actionItemRepository.findById(id)
        .orElseThrow(() -> new NotFoundException("해당 ActionItem이 없습니다. id="
+ id));

    // 제목 수정 (커서로 이름 바꿀 때)
    if (request.getTitle() != null && !request.getTitle().isBlank()) {
        actionItem.setTitle(request.getTitle());
    }

    // 기한 수정 (달력으로 바꿀 때)
    if (request.getDueDate() != null) {
        actionItem.setDueDate(request.getDueDate());
    }

    // 담당자 변경
    if (request.getAssigneeId() != null) {
        Member assignee = memberRepository.findById(request.getAssigneeId())
            .orElseThrow(() -> new NotFoundException("멤버를 찾을 수 없습니다."));
        actionItem.setAssigneeMember(assignee);
    }

    // @Transactional에 의해 자동 업데이트(Dirty Checking)
}
```

5. 프로젝트 태도 피드백 분석 엔진

역할

- 해당 프로젝트의 ActionItem 해결 정도를 최근 피드백(회의 참여도, 기존 Action Item 해결 정도)와 결합해 종합적인 피드백을 제시

핵심 코드

CommentService의 generateProjectAIAnalysis 및 FeedbackService의 getDailyBrief

설명

- 해당 프로젝트의 모든 ActionItem을 조회하고, 누적된 최근 피드백 데이터를 조회한다
- 이후 프롬프트에 따라 GPT 분석을 요청한다
- 완료된 분석은 이후의 피드백 분석 및 Daily Brief 조회에 다시 사용되므로, 저장된다
- Daily Brief는 저장된 피드백 데이터를 통한 데일리 코멘트, 예정된 회의목록, 오늘까지 할 일 목록을 조립해 보여준다.

```

@Transactional
public void generateProjectAIAnalysis(final Long memberId, final Long projectId)
{
    // [데이터 수집] 해당 프로젝트의 모든 ActionItem 조회
    List<ActionItem> projectTasks =
actionItemRepository.findByMeetingProjectId(projectId);

    long totalTasks = projectTasks.size();
    long completedTasks =
projectTasks.stream().filter(ActionItem::getCompleted).count();
    double successRate = totalTasks > 0 ? (double) completedTasks / totalTasks *
100 : 0;

    // [데이터 수집] 최근 피드백 내역 (누적 데이터)
    String feedbackHistory =
commentRepository.findTop5ByMemberIdAndRefTypeOrderByDateCreatedDesc(memberId,
"FEEDBACK")
        .stream()
        .map(Comment::getContent)
        .collect(Collectors.joining(" / "));

    // 3. GPT 분석 요청
    String systemPrompt = ""
    당신은 IT 팀빌딩 및 프로젝트 관리 전문가입니다.
    제공된 팀원의 성과 데이터를 바탕으로 개인의 태도 개선점과 팀의 현재 상태를
진단하세요.
    답변은 친절하면서도 객관적인 지표에 근거해야 합니다.
    """;

    String userPrompt = String.format("""
[분석 대상 데이터]
- 현재 프로젝트 Task 완료율: %.1f%% (%d개 중 %d개 완료)
- 최근 회의 피드백 히스토리: %s

[요청사항]
1. Task 완료율과 피드백 내용을 종합하여 이 팀원의 '프로젝트 참여 태도'를 분석하고
구체적인 개선 제안을 하세요.
2. 팀의 현재 협업 건전성을 '매우 좋음 / 보통 / 위험' 중 하나로 평가하고 그 이유를
짧게 설명하세요.
""",
        successRate, totalTasks, completedTasks,
        feedbackHistory.isEmpty() ? "누적된 피드백 데이터가 아직 없습니다." :
feedbackHistory
    );

    // [AI 호출 및 저장]
    String result = openAiClient.askGpt(systemPrompt, userPrompt);
    saveAiComment(memberId, "PROJECT_ANALYSIS", projectId, result);
}

```

데일리 AI 브리핑

FeedbackService.java 의 getDailyBreif 메서드

```
//대시보드: daily brief
public DailyBriefDTO getDailyBrief(final Long memberId) {
    // 1. 시간 범위 설정 (오늘)
    LocalDateTime startOfDay = LocalDate.now().atStartOfDay();
    LocalDateTime endOfDay = LocalDate.now().atTime(LocalTime.MAX);

    // 2. AI 데일리 코멘트 조회 (Comment 테이블 - DAILY_COMMENT)
    Comment comment =
commentRepository.findFirstByMemberIdAndRefTypeOrderByIdDesc(memberId,
"DAILY_COMMENT")
        .orElseThrow(() -> new NotFoundException("오늘의 AI 데일리 코멘트가
없습니다."));

    // 3. 오늘 예정된 회의 목록 조회 (Meeting 테이블)
    List<DailyBriefDTO.MeetingInfoDTO> meetings = meetingRepository
        .findByScheduledAtBetween(startOfDay, endOfDay).stream()
        .map(m -> DailyBriefDTO.MeetingInfoDTO.builder()
            .meetingId(m.getId())
            .agenda(m.getAgenda())
            .scheduledAt(m.getScheduledAt())
            .build())
        .toList();

    // 4. 오늘 마감인 투두 목록 조회 (ActionItem 테이블)
    List<DailyBriefDTO.TODOInfoDTO> todos = actionItemRepository
        .findByAssigneeMemberIdAndDueDateBetween(memberId, startOfDay,
endOfDay).stream()
        .map(a -> DailyBriefDTO.TODOInfoDTO.builder()
            .todoId(a.getId())
            .title(a.getTitle())
            .dueDate(a.getDueDate())
            .build())
        .toList();

    // 5. 최종 조립
    return DailyBriefDTO.builder()
        .id(comment.getId()) // Comment의 ID를 브리프 ID로 활용
        .date(LocalDateTime.now())
        .summary("오늘 예정된 회의는 " + meetings.size() + "건, 마감 투두는 " +
todos.size() + "건입니다.")
        .dailyComment(mapToDTO(comment, new CommentDTO()))
        .meetings(meetings)
        .todos(todos)
        .memberId(memberId)
        .activated(comment.getActivated())
        .build();
}
```

6. 일정/알림 관리

역할

- 사용자를 위한 일정 알림 전송
- 팀장에게만 공지 작성 권한 허용

핵심 코드

NotificationService의 createAnnouncementNotification 및 createFeedbackNotification

설명

- 팀장만 공지 작성이 가능하므로 공지 작성 전 팀장 역할을 맡은 사용자인지 검사하는 과정을 거친다
- 공지가 작성되면 팀의 모든 멤버에게 알림이 전송된다
- 사용자는 AI 피드백이 생성될 경우 알림을 받는다

```
//팀장 공지 알림
public void createAnnouncementNotification(final Long teamId, final Long
authorId,
                                     final String title, final String
message, final Long relatedId) {
    final Team team = teamRepository.findById(teamId)
        .orElseThrow(() -> new NotFoundException("팀을 찾을 수 없습니다."));

    final Member author = memberRepository.findById(authorId)
        .orElseThrow(() -> new NotFoundException("작성자를 찾을 수
없습니다."));

    // 팀의 모든 멤버에게 알림 생성
    team.getTeamMembers().forEach(teamMember -> {
        final Notification notification = new Notification();
        notification.setMember(teamMember.getMember());
        notification.setAuthor(author);
        notification.setTeam(team);
        notification.setType("ANNOUNCEMENT");
        notification.setTitle(title);
        notification.setContent(message);
        notification.setRelatedId(relatedId);

        notification.setActivated(true);

        notificationRepository.save(notification);
    });
}
```

```
}
```

```
//AI 피드백 알림 생성
```

```
public void createFeedbackNotification(final Long memberId, final String title,  
                                         final String message, final Long
```

```
relatedId) {
```

```
    final Member member = memberRepository.findById(memberId)  
        .orElseThrow(() -> new NotFoundException("멤버를 찾을 수 없습니다."));
```

```
    final Notification notification = new Notification();
```

```
    notification.setMember(member);
```

```
    notification.setType("FEEDBACK");
```

```
    notification.setTitle(title);
```

```
    notification.setContent(message);
```

```
    notification.setRelatedId(relatedId);
```

```
    notification.setActivated(true);
```

```
    notificationRepository.save(notification);
```

```
}
```

주요 기능 구현

본 시스템은 회의 데이터를 단순 저장하는 데 그치지 않고, 회의 내용을 실제 프로젝트 업무와 연결하기 위한 AI 기반 회의 분석 기능을 제공한다. 사용자는 회의 녹음 또는 음성 파일 업로드를 통해 회의 데이터를 입력할 수 있으며, 시스템은 이를 기반으로 STT 변환, 회의 요약, Todo 생성 기능을 수행한다.

특히 회의 분석 결과를 실제 프로젝트 관리 흐름으로 연결하기 위해 Todo 및 워크플로우 관리 기능(3. 프로젝트 관리 기능)을 제공한다. 기존 협업 도구가 회의와 업무를 별도로 관리하는 것과 달리, 회의 → Todo → 일정 → 피드백 흐름이 하나의 시스템 안에서 연결되도록 설계되었다.

주요 기능은 다음과 같다.

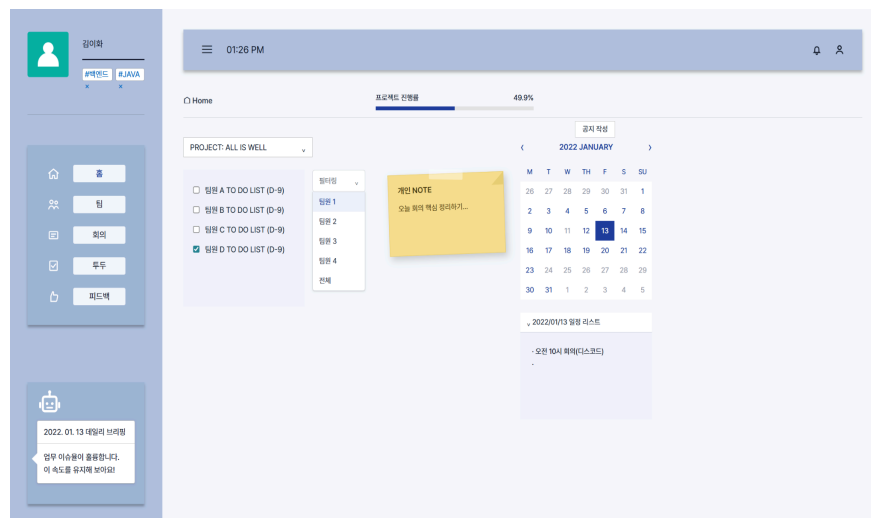
1. AI 기반 회의 분석 기능
 - 1.1. 대시보드
 - 1.2. 회의 녹음 및 파일 업로드
 - 1.3. STT 변환 및 화자 분리
 - 1.4. AI 회의 요약 및 Todo 생성
2. AI 피드백 제공 기능
 - 2.1. 회의별 피드백
 - 2.2. 주간 피드백(위클리 피드백)
3. 프로젝트 관리 기능
 - 3.1. 담당자 자동 매칭
 - 3.2. 일정 제안 및 Todo 관리

1.1. 대시보드

사용자가 로그인하면 가장 먼저 이동하는 프로젝트 대시보드 화면이다.

현재 진행중인 프로젝트의 진행 상태를 한눈에 확인할 수 있도록 구성하였다. 사용자는 현재 수행 중인 Todo 리스트, 프로젝트 진행도, 개인 메모, 캘린더 및 알림 정보를 확인할 수 있다.

좌측 사이드바를 통해 어떤 페이지에서도 회의, 회의 분석, 투두, 피드백 등의 큰 기능으로 바로 이동할 수 있다



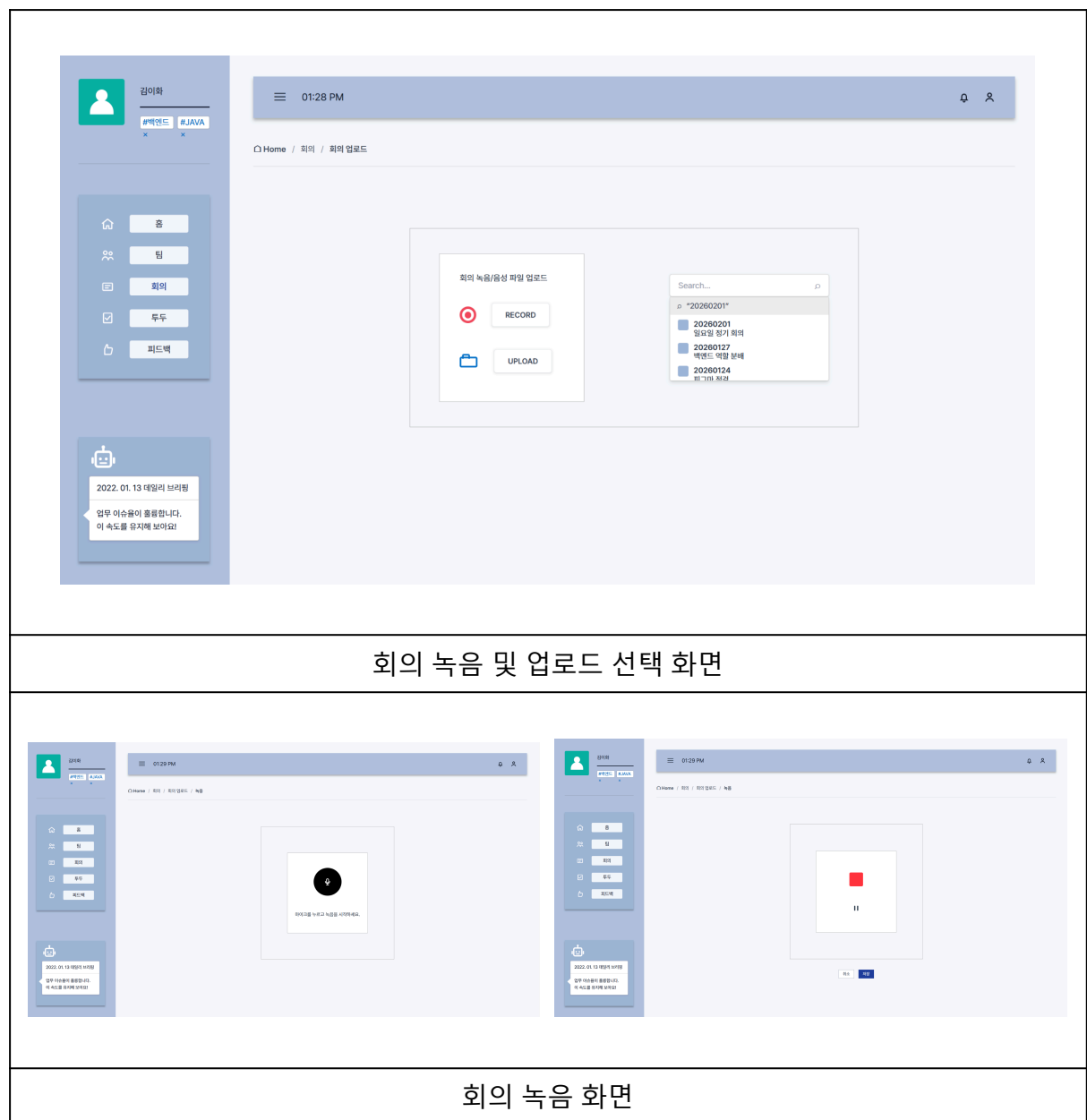
대시보드 화면

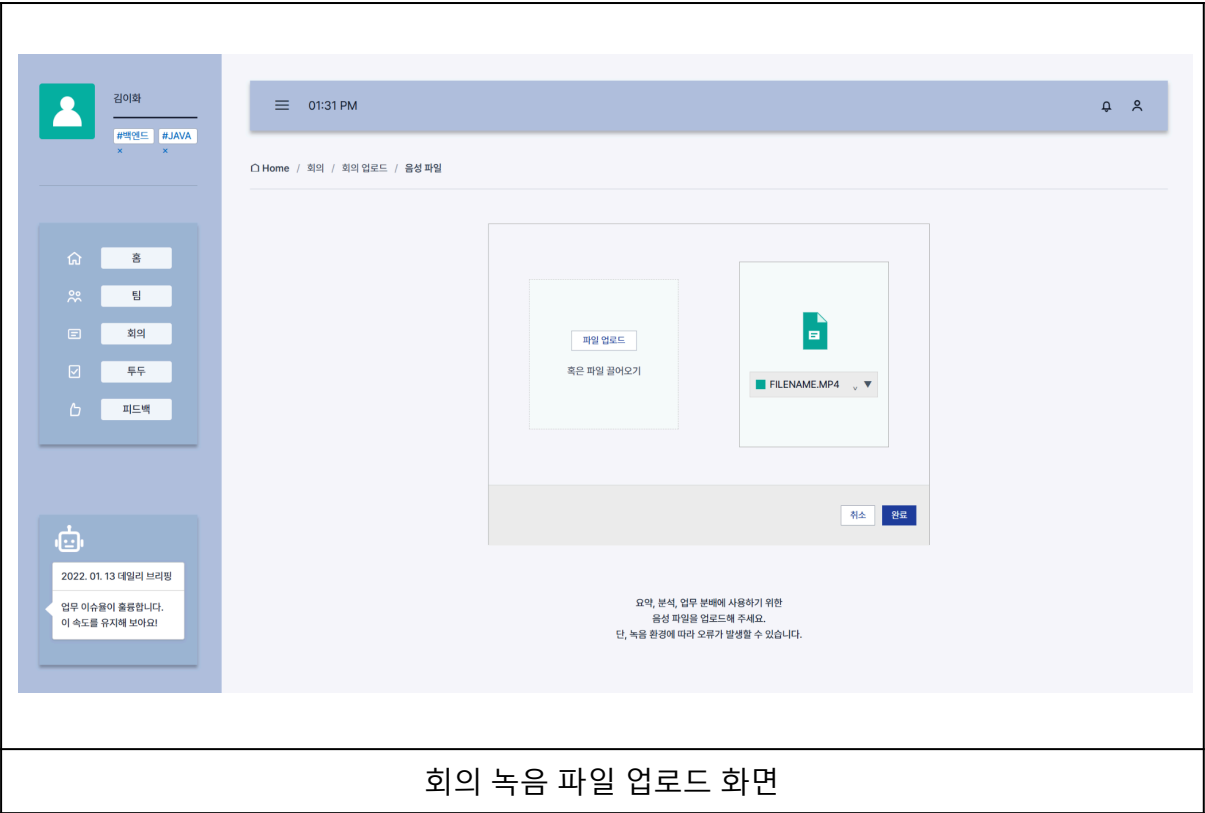
1.2. 회의 녹음 및 파일 업로드

사용자는 회의 업로드 페이지에서 직접 회의를 녹음하거나 기존 녹음 파일을 업로드할 수 있다. 녹음 기능은 사용자가 서비스 내부에서 바로 회의를 기록할 수 있도록 지원하며, 업로드 기능은 외부에서 녹음한 파일도 활용할 수 있도록 구성하였다.

녹음이 시작되면 시스템은 녹음 상태를 실시간으로 표시하며, 사용자는 녹음 종료 후 파일 저장을 진행할 수 있다. 또한 업로드된 음성 파일은 이후 STT 분석 단계로 전달된다.

이 과정은 회의 데이터를 수집하는 입력(Input) 단계에 해당하며, 회의 녹음본은 이후 AI 기반 회의 분석 기능의 기초 데이터로 활용된다.





1.3. STT 변환 및 화자 분리

업로드된 회의 파일은 Deepgram STT 기반 음성-텍스트 변환 과정을 거친다. 시스템은 변환된 회의 내용을 화자 단위로 분리하여 표시하며, 각 발언 내용을 구조화된 형태로 제공한다.

이를 통해 사용자는 회의 내용을 다시 듣지 않아도 텍스트 기반으로 회의 흐름을 빠르게 파악할 수 있으며, 이후 AI 분석 단계에서 발언 데이터와 참여 정보를 활용할 수 있도록 설계하였다.

특히 화자 단위 분리는 이후 담당자 자동 매칭 및 협업 피드백 생성 과정에서 핵심 데이터로 활용된다.

회의 화자 매칭 화면

1.4. 회의 요약 및 Todo 생성 기능

STT 변환이 완료되면 시스템은 GPT 기반 AI 분석을 수행하여 회의 내용을 자동으로 요약한다. 회의 요약 화면에서는 회의 핵심 내용, 주요 결정 사항, Action Item을 함께 제공하도록 구성하였다.

또한 회의 내용 중 실제 수행이 필요한 항목은 Todo 형태로 자동 생성된다. 사용자는 생성된 Todo 리스트를 확인하고 필요한 업무를 실제 프로젝트 Todo로 등록할 수 있다.

이를 통해 기존 협업 도구에서 사용자가 직접 회의록을 정리하고 업무를 생성해야 했던 비효율을 줄이고, 회의 → 업무(Task) 전환 흐름을 자동화하였다.

The screenshot displays the '회의 분석 화면' (Meeting Analysis Screen). The interface includes a sidebar on the left with a user profile for '김이화' and navigation buttons for Home, Team, Meeting, To-do, and Feedback. The main content area is titled '20251211 - 방학 프로젝트 회의' and contains a '회의 결정사항' (Meeting Decisions) list, an 'AI TODO' section with checkboxes, and an 'STT 변환 및 AI 요약 결과' (STT Conversion and AI Summary Results) section. The STT summary is divided into three segments: Segment 1 (Introduction), Segment 2 (Discussion on AI analysis), and Segment 3 (Conclusion). Below the summary is an 'ACTION ITEM EXTRACT' section. The bottom of the screen features a navigation bar with the text '회의 분석 화면'.

회의 분석 화면

2.1. 회의별 피드백

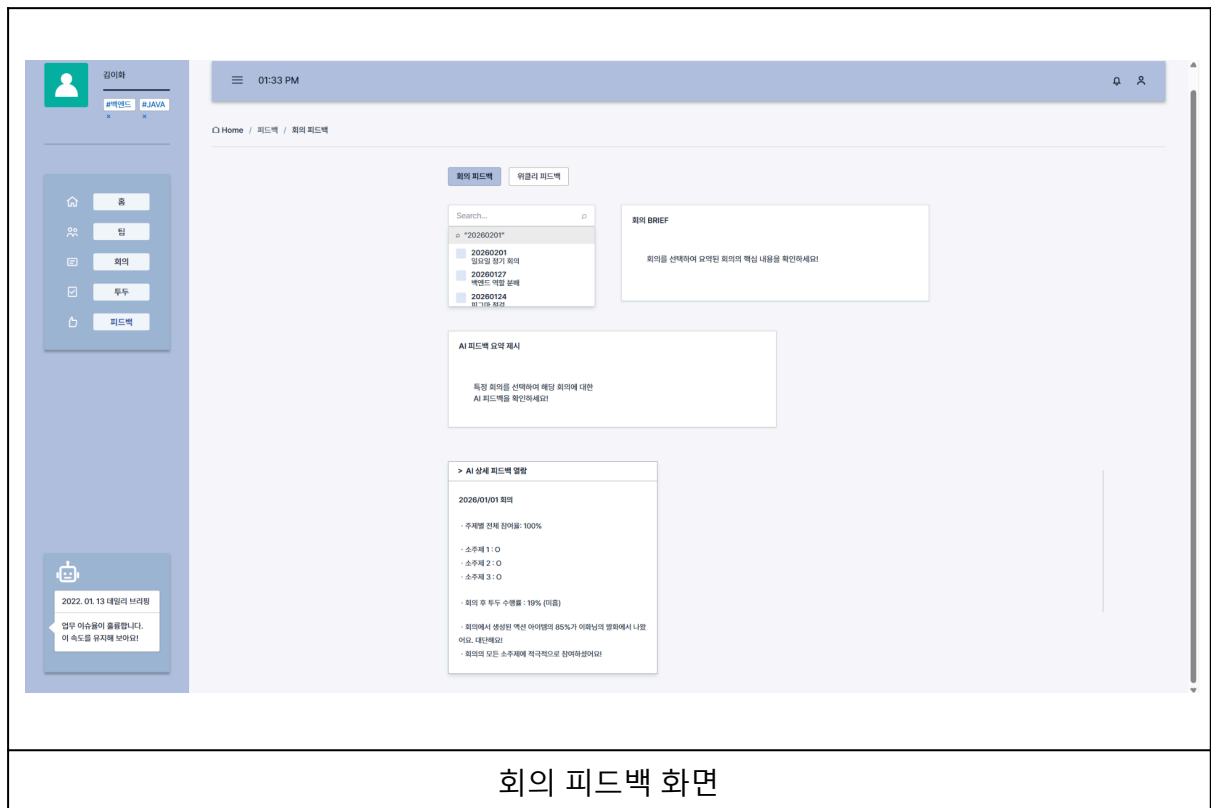
회의별 피드백 화면에서는 특정 회의에 대한 AI 분석 결과를 확인할 수 있다. 시스템은 회의 요약, STT 분석 결과, 생성된 Todo 데이터를 기반으로 회의 진행 상태를 분석한다.

특히 단순 발화량만을 기준으로 협업 상태를 판단하지 않고, 다음과 같은 요소를 종합적으로 고려하여 협업 피드백을 생성하도록 설계하였다.

- 회의 참여율
- 팀원별 발화 비중
- 발언의 Task 전환 여부
- Todo 수행률
- 프로젝트 진행 기여도

예를 들어 특정 사용자의 발언이 실제 Action Item 생성으로 이어지거나, 생성된 업무가 완료된 경우 높은 협업 기여도로 분석할 수 있다. 반면 발화량이 높더라도 실제 업무 수행과 연결되지 않는 경우 단순 참여 수준으로 판단하도록 설계하였다.

이를 통해 프로젝트 상태 분석은 논의 중심이 아니라 실행 중심의 분석의 형태를 띄게 된다.

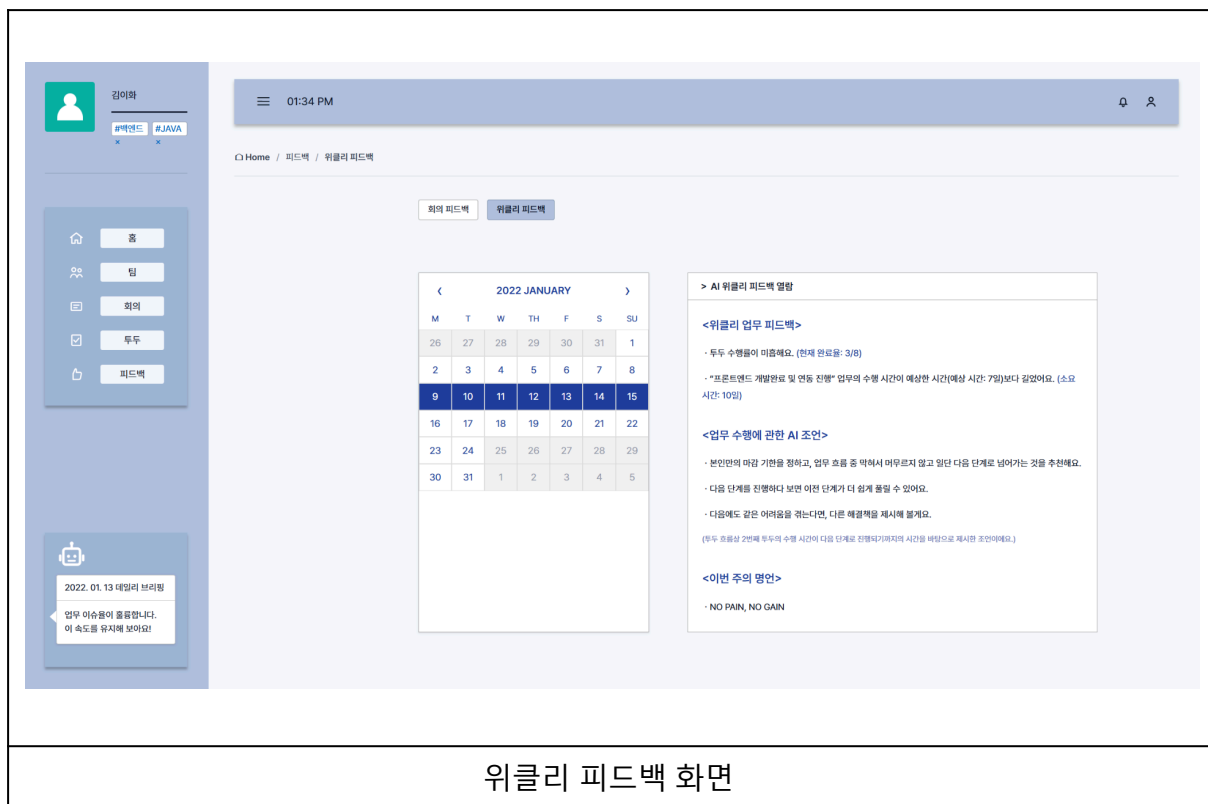


2.2. 주간(위클리) 피드백

주간 피드백 화면에서는 일정 기간 동안 누적된 회의 및 업무 데이터를 기반으로 프로젝트 협업 상태를 분석한다.

시스템은 주간 단위의 회의 참여율, Todo 완료율, 업무 분배 상태 등을 종합적으로 분석하여 사용자에게 프로젝트 진행 상태와 개선 방향을 제공한다. 또한 특정 인원에게 업무가 과도하게 집중되는 경우 협업 불균형 상태를 확인할 수 있도록 구성하였다.

이를 통해 사용자는 프로젝트 종료 이후가 아니라 진행 과정 중에도 자신의 협업 상태를 지속적으로 점검할 수 있다.

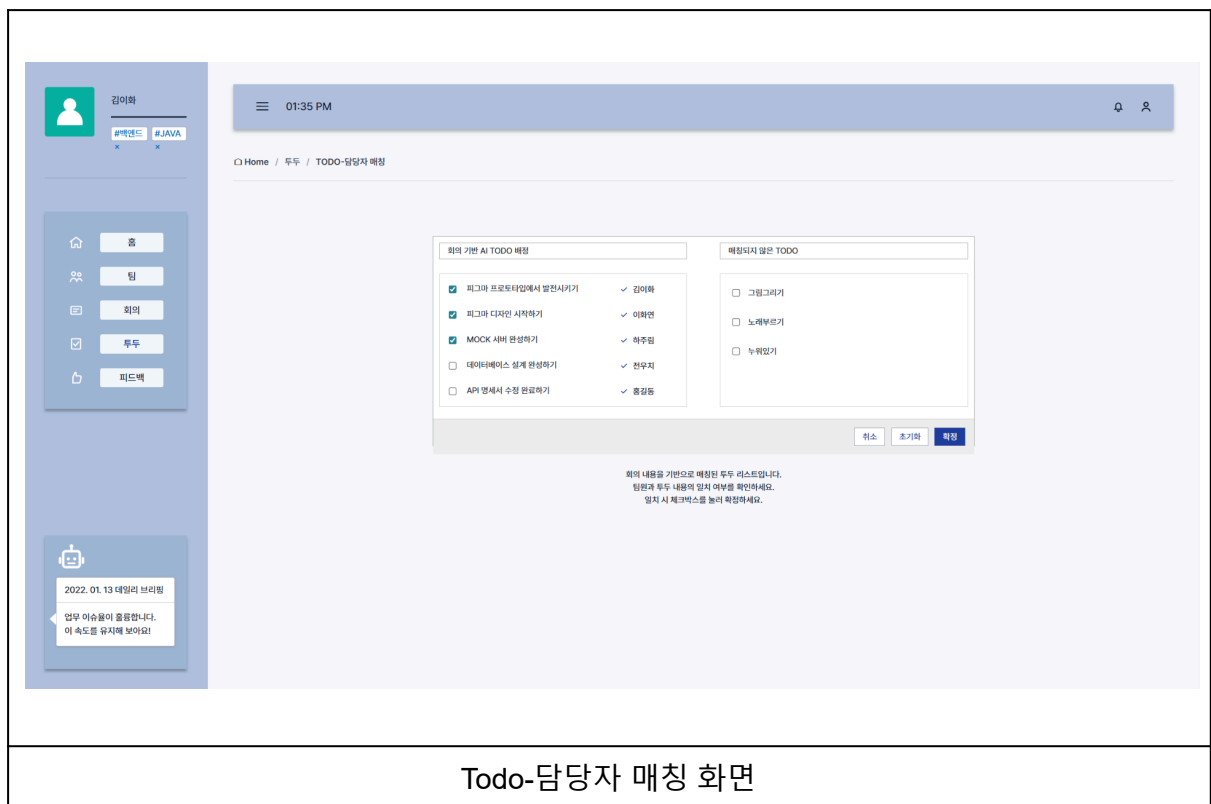


3.1. 담당자 자동 매칭

해당 기능을 통해 AI가 생성한 Todo는 회의 발언 데이터를 기반으로 담당자 자동 매칭이 가능하도록 하였다.

시스템은 회의 중 특정 업무와 관련된 발언을 수행한 사용자 또는 기존 유사 업무를 담당했던 사용자를 우선적으로 추천한다. 사용자는 추천된 담당자를 확인하고 실제 업무 담당자로 지정할 수 있다.

이를 통해 팀 내 업무 분배 과정에서 발생할 수 있는 혼선을 줄이고 보다 자연스러운 역할 분담이 가능하도록 하였다.



3.2. 일정 제안 및 Todo 관리

Todo 관리 화면에서는 프로젝트 진행 단계에 따라 업무를 구조적으로 관리할 수 있도록 구성하였다.

시스템은 생성된 Todo를 프로젝트 흐름에 맞게 분류하며, 사용자는 업무 상태(To Do / In Progress / Done)를 변경할 수 있다. 또한 AI는 업무 우선순위와 프로젝트 진행 흐름을 기반으로 일정 및 마감 기한을 제안한다.

이를 통해 사용자는 단순히 회의 내용을 기록하는 데서 끝나지 않고, 실제 프로젝트 진행 흐름을 지속적으로 관리할 수 있다.



김이화

#백엔드 #JAVA

홈

팀

회의

투두

피드백

2022. 01. 13 대일리 브리핑

업무 이슈들이 풀렸습니다.
이 속도를 유지해 보아요!

01:36 PM

Home / 팀 / 프로젝트 관리

프로젝트 진행 보드

KNOTE 협업프로젝트

TODO

회의 분석 결과 페이지 스크린샷 정리
진행중

보고서 기능 흐름 설명 작성
완료

백엔드 API 응답 형식 확인
백엔드

IN PROGRESS

회의 업로드 클로우 피팅러싱
진행중

회사 매칭 화면 구현
진행중

REVIEW

회의 분석 페이지 디자인 확인
완료

피드백 페이지 문구 검토
완료

DONE

피그마 화면 구성 완료
완료

중간발표 핵심 기능 범위 확정
완료

AI 일정 제안

날짜	시간	제안 일정
월	10:00	백엔드 API 응답 구조 확정 회의
수	14:00	프론트-백엔드 mock 연동 테스트
금	16:00	보고서 앱치 화면 최종 점검
토	11:00	회의 분석 클로우 시연 리허설

일정 추천 사유

이번 주는 회의 분석 클로우 구현과 보고서 앱치 장치가 가장 중요한 작업입니다.
따라서 팀원들의 작업 가능 시간을 기준으로 API 확인, 연동 테스트, 화면 점검 일정을 우선 배치했습니다.

가장 수락

Todo 관리 화면

기타

1. STT 엔진: Deepgram Nova-2 도입 및 최적화

기존 오픈소스 모델(Whisper)의 한계를 극복하고 실무적인 회의록 서비스를 구축하기 위해 **Deepgram Nova-2**를 채택하였다. 기술적으로 총 3가지 측면: 고속 추론, 정교한 화자 분리, 스마트 포매팅 기능에 근거하여 채택하였다.

1. 고속 추론
 - a. Whisper 대비 최대 20배 빠른 처리 속도로 사용자 대기 시간을 최소화 가능하다.
2. 정교한 화자 분리
 - a. 다중 화자가 겹쳐 말하는 상황에서도 화자를 분리하여 **Speaker ID**를 부여하는 능력이 탁월하다.
3. 스마트 포매팅
 - a. 한국어 특유의 숫자, 날짜, 문장 부호를 자연스럽게 교정하여 AI 분석의 기초 데이터 품질을 향상 가능하다.

REST API 기반의 비동기 처리를 통해 서버 자원 부담을 줄이고, 화자별 타임스탬프 데이터를 확보하여 '화자-사용자 매칭' UI를 구현하는 방식으로 시스템에 적용하였다.

2. AI 프롬프팅 고도화 과정 (Prompt Engineering)

단순 요약을 넘어 정형화된 업무 데이터를 추출하기 위해 프롬프트 수정 과정을 거쳤다. 프롬프트 수정 과정은 다음과 같은 3단계로 정리할 수 있다.

1. 기본

"회의 내용을 요약하고 할 일을 뽑아줘." 라고 명령하는 방법이다. 이와 같이 프롬프트를 작성하면 결과가 부정확하고 형식이 제각각인 문제가 발생한다.

2. 구조화

JSON 형식을 지정하고 **Action Items**의 필드를 정의하는 방법이다. 할 일은 뽑을 수 있으나, 마감 기한이나 담당 후보자가 누락되는 문제가 발생한다.

3. 지능형

Role Playing: "전문 프로젝트 매니저" 역할을 부여하는 방법이다. 이 방법을 통해 비즈니스 맥락 이해도를 증대시킬 수 있었다. 구체적인 방법은 다음과 같다.

- a. **Contextual Inference:** 대화 내용에 기한이 없더라도 업무 성격에 따라 마감 기한(dueDate)을 추천하도록 로직에 추가한다.
- b. **SDLC 기반 분류:** 할 일을 개발 공정(Phase 1~5)에 따라 자동 분류하도록 가이드라인을 제공한다.
- c. **JSON Mode 강제:** response_format을 지정하여 백엔드 파서(Parser)의 안정성을 확보한다.

이하는 3번: 지능형 Role Playing 방법에 따라 작성된 전체 프롬프트이다.

```
String systemPrompt =
    "너는 전문 회의 기록가이자 프로젝트 매니저야. 제공된 회의 스크립트를 분석해서
    'Action Items(할 일)'를 추출해.\n\n" +
    "##지침##\n" +
    "1. ##사실 근거## 오직 회의 스크립트에서 언급된 할 일만 추출해. 추측하거나
    새로운 일을 만들지 마.\n" +
    "2. ##마감 기한(dueDate) 추천## 대화 맥락상 언급된 기한이 있다면 그것을
    사용하고, 없다면 업무의 시급성을 판단해 추천해줘.\n" +
    "    - 오늘 회의 날짜 기준으로 추천하되, 형식은 'YYYY-MM-DDTHH:mm:ss'로
    고정해.\n" +
    "3. ##화자 매칭## 해당 할 일을 하기로 했거나 언급한 화자를
    'suggestedSpeaker' 필드에 정확히 적어줘.\n" +
    "4. ##형식## 반드시 아래 JSON 형식으로만 응답해.\n\n" +
    "{\n" +
    "  \"title\": \"회의 제목\",\n" +
    "  \"summarySegments\": [\"요약 문장1\", \"요약 문장2\"],\n" +
    "  \"actionItems\": [\n" +
    "    {\n" +
    "      \"title\": \"할 일 제목\",\n" +
    "      \"memo\": \"상세 내용\",\n" +
    "      \"suggestedSpeaker\": \"SPEAKER_0\",\n" +
    "      \"dueDate\": \"2026-05-14T18:00:00\"\n" +
    "    }\n" +
    "  ]\n" +
    "}\n" +
    "+ \"5. ##Phase 분류## 각 할 일의 성격에 따라 아래 기준을 참고하여 'phase'
    필드에 1~5 사이의 숫자를 부여해.\n" +
    "+ \"    - 1 (Ideation): 아이디어 회의, 요구사항 정리 관련\n" +
    "+ \"    - 2 (Design): 설계, UI/UX, DB 모델링 관련\n" +
    "+ \"    - 3 (Development): 실제 코드 작성, API 구현 관련\n" +
    "+ \"    - 4 (Testing): 버그 수정, 테스트, 리팩토링 관련\n" +
    "+ \"    - 5 (Launch): 배포, 문서화, 최종 발표 관련\";
```

3. 팀원 기여도 및 피드백 생성 엔진

팀 프로젝트의 건강한 협업을 위해 발화량, 투두 수행률 등을 분석하여 AI 피드백을 제공하는 엔진이다. 이 프롬프트는 회의 데이터와 팀원별 성과 데이터를 취합하여 분석할 때 사용되며, 아래와 같은 형식으로 이루어져 있다.

```
String feedbackPrompt =
    "너는 초보 팀프로젝트에서 리더를 맡은 유능한 사람이야. 아래 제공된  

    데이터(회의 발화 내용, TODO 수행 통계)를 바탕으로 팀원 각자에게 격려와 개선  

    방향을 담은 피드백을 작성해줘.\n\n" +
    "**분석 지표:**\n" +
    "1. 발화 적극성: 회의 중 의견 제시 빈도 및 아이디어 기여도\n" +
    "2. 실행력: 지난주 대비 TODO 완료 상태 및 기한 준수 여부\n" +
    "3. 협업 태도: 타인의 의견에 대한 반응 및 지원 의사 표명\n\n" +
    "**출력 규칙:**\n" +
    "- 팀원별로 '강점'과 '보완점'을 나누어 긍정적인 톤으로 작성할 것.\n" +
    "- 반드시 JSON 형식으로 출력할 것.\n\n" +
    "{\n" +
    "  \"teamFeedback\": \"팀 전체에 대한 총평\", \n" +
    "  \"memberFeedbacks\": [\n" +
    "    {\n" +
    "      \"memberName\": \"이름\", \n" +
    "      \"score\": 95, \n" +
    "      \"analysis\": \"적극적인 기술 제안이 돋보임\", \n" +
    "      \"advice\": \"다음 단계에서는 팀원들의 일정 배분에도 관심을  

    가져보세요.\", \n" +
    "    }\n" +
    "  ]\n" +
    "}";
```